



PROFILE

임진우

Backend Developer

안정성과 효율을 함께 고민하며,
문제를 끝까지 해결하는 백엔드 개발자입니다.

✉ gart09@naver.com

🌐 github.com/gart09

학력

2014.03 ~ 2026.08

서울시립대학교 컴퓨터과학부

입학 및 졸업

2010.03 ~ 2013.02

백석고등학교

졸업

경력

2026.01 ~ 2026.02

풀어비스 검은사막팀 프로그래밍 인턴

2024.08 ~ 2026.01

서울시립대학교 산학협력단 검수전담 직원

2015.05 ~ 2020.05

리그 오브 레전드 프로게이머

2019.05 ~ 2020.05 LPL EDG

2018.12 ~ 2019.05 LPL V5

2015.05 ~ 2018.12 LPL LGD

외부 활동

2026.04 ~ 2026.10

유플러스 유레카 부트캠프

백엔드 과정

자격증 · 어학

2026.09.11

정보처리기사

2025.12.12

SQLD

2026.03.08

TOEIC Speaking IM3

기술 스택

Language

Java

진행한 프로젝트의 메인 언어로 사용하며 백엔드 로직 전반을 구현

C++

게임 프로그래밍을 진행할 때 메인 언어로 사용, C++ STL 및 기본 문법에 익숙

Database

MySQL

스키마 설계부터 쿼리 작성까지 관계형 데이터베이스 전반을 다뤄봄

Framework

Spring Boot

REST API와 서비스 로직 구현 경험

Spring Batch

대용량 데이터(수백만 건) 비동기 정합성 검증 파이프라인을 Job/Step 단위로 직접 설계·구현

Tools

GitHub

ISSUE/PR 템플릿과 PR 컨벤션을 적용해 협업 기본 도구로 사용

Docker

Docker Compose로 기초 인프라 환경을 조성하고, Docker 명령어로 손쉽게 DB를 초기화하는 방식을 적용

Ace

선착순 쿠폰 발급 시스템

Java

Spring Boot

Spring Batch

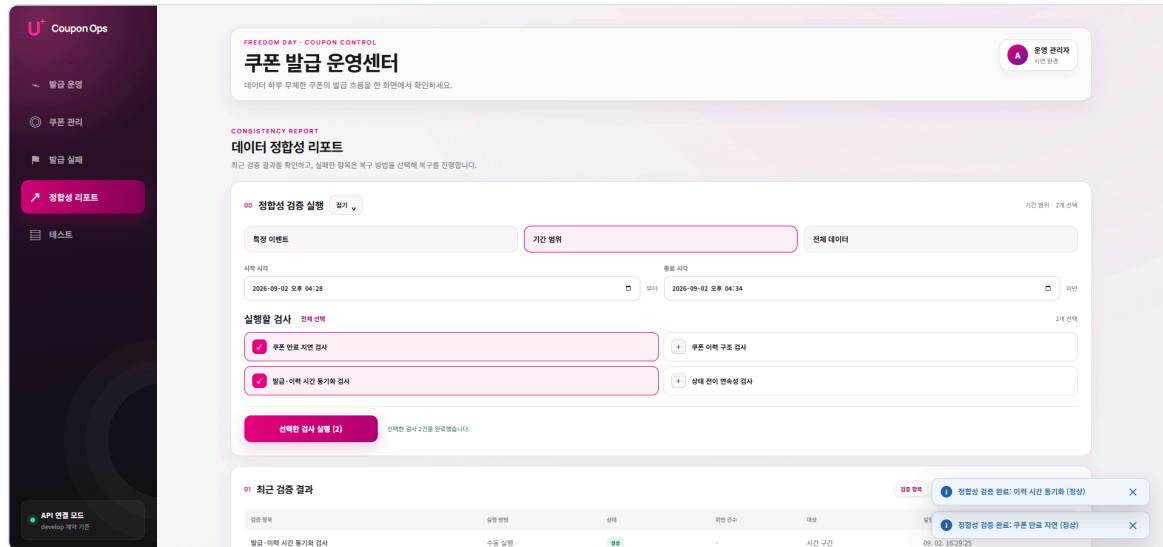
MySQL

Redis

Docker Compose

LG유플러스 유레카 종합 프로젝트 최우수상

2026.08.06 — 2026.08.31


github.com/URECA-Ace

프로젝트 소개

대규모 동시 요청에서도 안정적인 쿠폰 발급과 사후 데이터 검증·복구를 지원하는 선착순 쿠폰 발급 시스템

프로젝트 목표

동시 요청 상황에서 쿠폰의 초과 발급을 방지하고 데이터 정합성 보장

DB에 저장된 데이터를 검증할 수 있는 사후 처리 성격의 정합성 검증 시스템 구현

내가 기여한 부분

협업을 위한 기초 인프라 환경 조성

ISSUE/PR 템플릿 , Git Hook , Docker Compose 등
협업에 필요한 기초 인프라 환경을 조성

정합성 검증 공통 로직 구현

정합성 검증 공통 인터페이스와 실행 파이프라인을 구현
재고 정합성 검증을 레퍼런스로 구현
팀원들이 동일한 구조로 검증 로직을 개발할 수 있도록 함

Spring Batch 를 활용한 비동기 검증 파이프라인 구현

대규모 데이터 정합성 검증을 체크 단위로 처리
실패 시 중단 지점부터 재개할 수 있는 비동기 파이프라인을 구현

정합성 위반 건 복구 정책 공통 로직 구현

정합성 복구 정책 공통 인터페이스와 실행·이력 저장·재검증 파이프라인을 구현
관리자 확인 후 필요한 복구 수행
재고 정합성 검증 위반 복구 정책을 레퍼런스로 제공

Ace

선착순 쿠폰 발급 시스템

주요 구현 내용

I 정합성 검증 파이프라인

공통 인터페이스와 Spring Batch 기반 비동기 검증 구조

3명의 팀원이 정합성 검증 파트를 담당

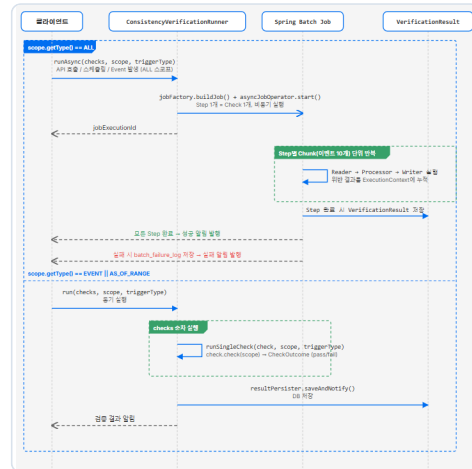
일반된 구조로 개발할 수 있도록 공통 인터페이스 설계

프로젝트가 완성되지 않은 시점

정합성 검증의 호출 위치와 실행 시점이 확정되지 않은 상태

코드 구현과 실행 시점을 분리

Runner 에 필요한 매개변수를 전달하면 해당 검증 로직이 실행되도록 구현



정합성 검증 시퀀스 다이어그램

I 검증 실행 공통 인터페이스

검증을 실행하는 곳에서는 Runner에 다음 3가지 인자만 전달하면 검증을 실행할 수 있는 구조로 구현

```
runner.run(selectedChecks, scope, TriggerType.ON_DEMAND);
```

checks

실제로 검증할 정합성 검증 객체
ex) Stock, Structural, StateMachine...

scope

검증을 실행할 범위를 결정하는 매개변수
ex) 특정 쿠폰 이벤트, 시간 구간, 전체

triggerType

검증 실행 방식을 결정하는 매개변수
ex) 스케줄러, 수동 실행, 복구 후 재검증

I Spring Batch 기반 비동기 검증

ALL 스코프는 한 번에 수백만 건을 검사하므로 시간이 많이 소요

요청과 검증 실행을 분리하고 Spring Batch Job으로 처리하도록 구현

마찬가지로 ConsistencyVerificationRunner 를 통해 인자만 건네면 쉽게 실행

```
runner.runAsync(allChecks, Scope.all(to), TriggerType.SCHEDULED);
```

ConsistencyBatchJobFactory 가 요청받은 검증 1개당 Step 1개씩 동적으로 조립해 Job을 구성

Step 구성 요소

01

EventIdPageReader

- event_id 기준 keyset pagination으로 coupon_event 를 10개씩 조회
- ItemStream 으로 마지막 event_id 를 저장해, 비정상 종료 후 재시작 시 해당 지점부터 이어서 읽음

02

ConsistencyCheckItemProcessor

- Reader가 읽은 페이지(coupon_event 10 개)로 해당 Step의 검증을 실행
- 실행 도중 오류가 발생하면 커밋된 페이지 이후부터 재실행

03

CheckResultAccumulatorWriter

- 페이지별 위반 건을 즉시 저장, 위반 건수는 Step 종료까지 누적
- ItemStream 으로 누적 위반 건수도 보존해 재시작 후에도 이전 집계 유지
- Step 종료 시 StepCompletionListener 가 VerificationResult 를 최종 저장

Ace

선착순 쿠폰 발급 시스템

주요 구현 내용

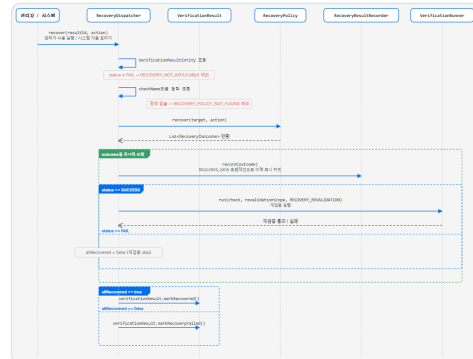
I 정합성 검증 복구 정책

복구 실행부터 이력 저장 · 재검증까지의 공통 파이프라인

정합성 검증마다 위반 규칙과 복구 방식은 상이
하지만 복구를 실행하고 결과를 관리하는 흐름은 공통으로 필요

복구 정책 파이프라인은 일관되게 실행
검증별 실제 복구 로직은 각 팀원이 ConsistencyRecoveryPolicy 로 구현

복구 정책 실행 · 복구 이력 저장 · 성공 시 재검증 ·
VerificationResult 상태 갱신 흐름을 공통화



정합성 검증 복구 정책 시퀀스 다이어그램

I 복구 실행 공통 인터페이스

트리거 종류(관리자 실행, 자동 실행)에 상관없이 ConsistencyRecoveryDispatcher.recover() 하나를 거쳐 복구하도록 구현

```
List<RecoveryResult> results = dispatcher.recover(resultId, action);
```

resultId

복구 대상인 FAIL 상태의 정합성 검증 결과
FAIL 상태가 아니면 즉시 예외로 거부

action

관리자가 선택한 복구 방식(RecoveryAction)
검증별 정책에 맞는 실제 복구 로직을 선택

I 복구 진행 과정

01

정책 실행

- 위반 VerificationResult 의 checkName 으로 수행한 검증 식별
- 하나의 검증에서 여러 coupon_event 가 개별적으로 위반될 수 있음
- 해당 검증의 복구 정책을 찾아 각 위반 건에 대한 복구 수행
- 건별 복구 결과를 List<RecoveryOutcome> 로 반환

02

이력 저장

- ConsistencyRecoveryDispatcher 가 List<RecoveryOutcome> 를 순회
- 각 RecoveryOutcome 의 복구 성공·실패 여부와 관계없이 모든 복구 시도를 recovery_result 테이블에 저장
- 건별 REQUIRES_NEW 트랜잭션으로 결과를 즉시 커밋해 이력을 보존

03

재검증 및 상태 갱신

- 복구 성공 시 동일한 검증을 자동으로 다시 실행
- 재검증까지 통과한 경우에만 최종 복구 성공으로 인정하고 상태를 갱신

Ace

선착순 쿠폰 발급 시스템

고찰 · 개발 이슈

위반 건수가 많아지면 검증 결과 저장이 통째로 실패

대량의 위반 데이터를 하나의 컬럼에 저장하며 발생한 MySQL 정렬 메모리 오류

문제 상황

VerificationResult의 diff_detail 컬럼 안에 위반 내역 메타데이터를 하나의 sample 키로 저장

위반 건수가 많아지면 컬럼 데이터가 커질 수 있음을 사전에 인지
하지만 대량 위반은 발생하지 않을 것이라 판단, 기존 구조를 유지

테스트 중 Redis와 로컬 환경의 시계가 어긋나면서 정상 데이터가 대량 위반으로 판정되어, 위반 1만 건이 한 번에 쌓이는 상황 발생

MySQL 조회 과정에서 대용량 diff_detail 값이 정렬 대상에 포함
정렬 메모리가 부족해져 Out of sort memory 오류 발생

```
private Map<String, Object> buildDiffDetail(List<Map<String, Object>> violations) {
    Map<String, Object> diff = new LinkedHashMap<>();
    diff.put("violationCount", violations.size());
    diff.put("sample", violations.stream()
        .limit(SAMPLE_LIMIT)
        .map((Map<String, Object> row) -> Map.of(
            K1 "eventId", row.get("event_id"),
            K2 "totalStock", row.get("total_stock"),
            K3 "issuedQuantity", row.get("issued_quantity"),
            K4 "remainingStock", row.get("remaining_stock"),
            K5 "actualActiveCount", row.get("actual_active_count")
        ))
        .toList());
    return diff;
}
```

기존 diff_detail 저장 구조

해결

위반 사항을 verification_violation 테이블에 저장
위반 1건을 행 1개로 분리

diff_detail은 각 검증이 필요한 개별 메타데이터만 저장하도록 역할을 축소

위반 건수가 늘어나더라도 검증 결과 저장이 실패하지 않도록 개선
유형·대상별 조회 및 복구 정책 연결이 가능해짐

```
private Map<String, Object> buildDiffDetail(List<Map<String, Object>> violations) {
    Map<String, Object> diff = new LinkedHashMap<>();
    diff.put("violationCount", violations.size());
    return diff;
}

private List<Violation> buildViolations(List<Map<String, Object>> violations) {
    return violations.stream()
        .map((Map<String, Object> row) -> new Violation(
            ViolationTargetType.EVENT,
            ((Number) row.get("event_id")).longValue(),
            Map.of(
                K1 "eventId", row.get("event_id"),
                K2 "totalStock", row.get("total_stock"),
                K3 "issuedQuantity", row.get("issued_quantity"),
                K4 "remainingStock", row.get("remaining_stock"),
                K5 "actualActiveCount", row.get("actual_active_count")
            ))
        .toList();
}
```

개선된 위반 데이터 저장 구조

배운 점

문제 가능성을 인지했지만, 발생 가능성이 낮다고 판단해 구조 개선을 후순위로 미룸

가장 큰 문제는 실제 이슈 발생 시 어떤 영향으로 이어질지 충분히 검토하지 않았다는 점

당시 MySQL에 대한 이해가 부족해, 정렬 과정에서 큰 컬럼으로 인해 오류가 발생할 수 있다는 점을 고려하지 못함

이슈의 발생 가능성뿐 아니라 발생 시 영향까지 확인한 후, 문제의 크기를 판단해 구조 변경 여부를 논의하는 것이 중요함을 깨달음

고찰 · 협업 이슈

로컬 Git Hook으로 커밋 메시지 컨벤션 검증

일관된 Git 히스토리를 만들기 위해 커밋 시점의 규칙 검증을 도입

도입 배경

이전 프로젝트에서 팀원마다 커밋 방식이 달랐음
메시지의 일관성이 떨어져 커밋 히스토리를 관리하는 데 불편함을 겪음

커밋 컨벤션을 문서화하는 것만으로는 일관성을 유지하기 어렵다고 판단
문서화에 그치지 않고 커밋 단계에서 규칙을 검증할 필요성을 느낌

구현 방법

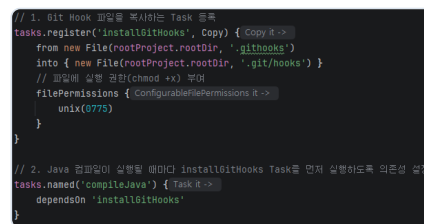
커밋 메시지 컨벤션을 `.git/hooks` 아래의 `commit-msg` 파일로 검증하도록 구현

하지만 `.git` 은 저장소 메타데이터 폴더
git 버전 관리 대상이 아니므로, Hook 스크립트를 커밋해 팀원과 공유할 수 없었음

gitHooks 폴더에 commit-msg 를 두고, 프로젝트 실행 시 해당 파일을 .git/hooks/ 로 복사하는 Gradle Task를 구현



커밋 메시지 검증



Hook 설치 Gradle Task

[문제] 로컬 Git Hook은 강제력이 부족

저장소를 새로 받은 팀원이 프로젝트를 실행해야만 로컬 Git Hook이 설치됨
팀원이 프로젝트를 한 번도 실행하지 않으면 검증이 동작하지 않음

git commit --no-verify 로 Git Hook을 무시할 수 있음
컨벤션을 지키지 않아도 커밋 가능

PRO 원격 저장소에서 머지될 때는 로컬 Git Hook이 동작하지 않음

결과적으로 PR 머지 시점에는 커밋 메시지가 다시 일관성 없이 남게 됨



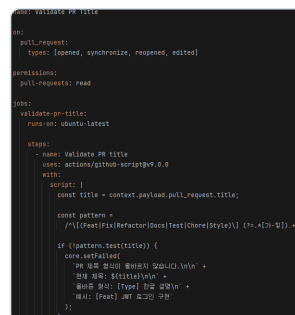
일관성 없이 남은 커밋 메시지

■ 개선 방향

로컬이 아닌 원격 저장소에서 규칙을 검증하도록 GitHub Actions를 도입

PR 생성·업데이트 시 커밋 메시지 컨벤션을 검사
검증을 통과한 PR만 머지되도록 구성

현재 진행 중인 프로젝트에도 적용
일관된 커밋 메시지가 유지되도록 강제할 수 있었음



GitHub Actions 스크립트



현재 프로젝트